Oh-my-os 项目进度报告

从零实现的 x86 操作系统内核

项目名称: Oh-my-os — 自研单用户命令行操作系统
目标硬件: ASUS VivoBook 15 X542UF (i5-8250U, 8GB RAM, SATA SSD)
开发语言: C++ / x86 Assembly
报告日期: 2026 年 6 月 2 日
当前阶段: M5 完成，M6 开发中

总体进度: 50%

done!30M0–M5 已完成 (50%)	inprogress!20M6	M7–M11 待完成

M0 已完成 M1 已完成 M2 已完成 M3 已完成 M4 已完成 M5 已完成 M6 进行中 ○ M7 待开始 ○ M8 待开

开发者: Wangmj
仓库分支: Wangmj
最新提交: `5e7f319` M5: Syscall Linux ABI 兼容 + Shell 扩展

Contents

1 项目概述

Oh-my-os 是一个从零开始实现的 x86 单用户命令行操作系统，最终目标是在华硕 VivoBook 15 X542UF 实机上运行第三方开源软件（vim、gcc、curl）。内核运行于 32 位保护模式，系统调用接口兼容 Linux i386 ABI（int 0x80），目标移植 musl libc 以获得完整 C11 标准库支持。

1.1 目标硬件规格

项目	规格
型号	ASUS VivoBook 15 X542UF
CPU	Intel Core i5-8250U @ 1.60GHz (Kaby Lake R, 4核8线程)
架构	x86_64（内核运行于 32 位保护模式）
内存	8 GB DDR4
硬盘	111.8 GB SATA SSD (AHCI)
显卡	Intel UHD Graphics 620 + NVIDIA MX130
固件	Legacy BIOS
键盘	PS/2 (AT Translated Set 2)

Table 1: 目标硬件规格

1.2 核心设计原则

原则 A POSIX 兼容的系统调用接口 — 系统调用编号、参数、返回值语义兼容 Linux i386 ABI（int 0x80），未实现调用返回 -ENOSYS。确保 musl libc 无需修改即可运行。

原则 B先自写 libc 验证，最终移植 musl — 开发阶段用自写简化 libc（~500 行）快速验证系统调用链路；后移植 musl 获得完整 C11 标准库。

原则 C一切皆文件的 VFS 抽象 — 磁盘文件、设备节点、管道、网络 socket 统一为文件描述符，通过 struct file_ops 操作。

2 总体进度概览

本项目共规划 12 个里程碑（M0–M11），目前已完成 M0 至 M5，M6（文件系统）正在开发中。总 50%。

3 已完成阶段详情

3.1 M0: 512B MBR Demo

· 手写 512 字节 MBR 启动扇区（以 0x55AA 结尾）

· BIOS 加载到 0x7C00，在 VGA 文本模式显示 ”Hello OS!”

· 验证了从源码到实机运行的完整工具链

阶段	名称	状态	关键成果
M0	Demo (512B MBR)	已完成	MBR 输出 ”Hello OS!” 到 VGA
M1	Bootloader	已完成	保护模式切换, A20, GDT, 加载内核 ELF
M2	中断系统	已完成	IDT + 8259A PIC + PIT + PS/2 键盘
M3	内存管理	已完成	E820 + 位图 PMM + 二级页表 + kmalloc + Demand Paging
M4	进程管理	已完成	PCB + RR 调度 + fork/exit/wait + Ring3 用户态
M5	系统调用兼容	已完成	Linux ABI 兼容 + Shell 扩展
M6	磁盘驱动+文件系统	进行中	AHCI/PCI 驱动开发中, FAT32 + VFS
M7	用户态 + Shell	○ 待开始	ELF 加载器 + 简化 libc + 交互式 Shell
M8	IPC + 管道	○ 待开始	pipe + dup2 + 信号
M9	网络栈	○ 待开始	TCP/IP + RTL8111 以太网驱动
M10	ACPI 电源管理	○ 待开始	关机/重启
M11	SMP 多核	○ 待开始	4 核 8 线程并行调度

Table 2: 里程碑进度总览

3.2 M1: Bootloader + 保护模式

实现内容:

- **MBR**（boot/mbr.S，~110 行）：INT 0x13 AH=0x42 扩展读加载 Stage2 到物理地址 0x500
- **Stage2**（boot/stage2.S，~400 行）：A20 三级 Fallback（BIOS → 键盘控制器 → Fast A20）→ GDT 平坦模型 → CR0.PE=1 → 加载内核 ELF 到 1MB
- **内核入口**（kernel/arch/x86/boot.S，~70 行）：设栈 → 清零 BSS → C++ 全局构造器 → 调用 kernel_main()
- **VGA 输出**：printk 格式化打印到 0xB8000 显存

关键技术决策:

- Stage2 加载到物理 0x500，链接基址 = 物理地址，解决 GDT 地址重定位问题
- 实模式下 INT 0x13 读盘 → 保护模式后 REP MOVSL 复制到 1MB

实机踩坑（4 个）： Stage2 日志闪现后重启（GDT 基址错误）；磁盘读失败（硬编码盘号）；保护 ≠ 物理地址）；屏幕乱码（清屏用 NULL 而非空格）。

3.3 M2: 中断系统

实现内容:

- **IDT**：256 个中断门描述符，前 32 个 CPU 异常 + 16 个 IRQ + int 0x80 系统调用门（DPL=3）
- **8259A PIC**：ICW1–ICW4 初始化序列，IRQ 0–15 重映射到中断向量 0x20–0x2F
- **ISR 汇编桩**（isr_stubs.S，160 行）：48 个桩 + 公共分发器，保存全部寄存器，构造栈帧传 C++ handler
- **PIT 8253**：方波发生器 Mode 3，1193180 / 11931 = 100.006 Hz（误差 0.006%）
- **PS/2 键盘**：AT Set 2 协议状态机（0xF0 break / 0xE0 扩展 / Shift/CapsLock），扫描码→ASCII 映射，256 字节环形缓冲区
- **系统调用框架**：int 0x80 陷阱门，预留 Linux ABI 兼容接口

实机踩坑（4 个）： jiffies 不增长（ISR 栈帧偏移 40B 错误）；PIT 无输出（I/O 延迟不足）；Shift “粘住”（break code 未处理）；屏幕字符异常（清屏值错误）。

3.4 M3: 内存管理

实现内容:

- **物理内存探测**：Stage2 实模式调用 BIOS INT 0x15 (AX=0xE820)，循环收集内存 map 写入 0x2000 缓冲区，支持 ACPI 3.0 24 字节条目
- **物理页框分配器（PMM）**：位图法，每 bit 代表 4KB 页。两次遍历 E820：Pass 1 找最大物理地址以确定位图大小；Pass 2 标记 type=1 区域为空闲。E820 无效时回退 Safe-Zone（1MB–128MB）
- **二级页表（VMM）**：10-10-12 分页，identity-map 0–4MB 确保内核切换后仍可访问。vmm_map / vmm_unmap_page / vmm_virt_to_phys API，PDE 不存在时自动分配新页表
- **内核堆（kmalloc/kfree）**：整页分配器，堆区域 0xD0000000–0xE0000000（256MB）。块头含 magic（0x4B4D414C），krealloc 支持
- **Demand Paging**：#PF handler（INT 0x0E）读取 CR2 获取故障地址 → 分配物理页 → 映射 → iret 重试

实机踩坑（6 个）： PIC 初始化后立即异常（io_wait() 写 port 0x80 不兼容华硕 EC）；E820 数据全乱码（buffer 与内核加载区域重叠）；printk %08x 不支持导致参数错位；QEMU 正常实机崩溃（内核扇区数不足截断）；Shift 不生效（Set 1 vs Set 2 扫描码混淆）；Stage2 E820 count 显示异常（print_hex 低 4 位 bug）。

3.5 M4: 进程管理

Part 1 — 内核线程调度（Commit: `1bc51cb`）:

- **PCB 数据结构**：pid, state（READY/RUNNING/BLOCKED/ZOMBIE）, cpu_context（esp/ebp/ 内核栈, cr3, parent_pid, exit_status
- **调度器**：Round-Robin 环形就绪队列，协作式 proc_yield() + PIT 100Hz 抢占式 proc_tick()
- **上下文切换**（switch.S）：保存/恢复 5 个 callee-saved 寄存器，switch_to(prev, next)

Part 2 — 用户态进程 + 系统调用（Commit: c555a20）：

· **GDT + TSS**：6 段 GDT（KCODE/KDATA/UCODE/UDATA/TSS），TSS.esp0 每次上下文切换前更

· **Ring3 切换**：ring3_trampoline 设用户 DS → iret → Ring3 用户代码

· **系统调用**：int 0x80 → isr_syscall pushal → syscall_handler → 写返回值到 eax 槽 → iret

· **fork()**：深拷贝页目录（内核 PDE 共享，用户 PDE 全量复制），全新构建子进程内核栈（iret 帧 + pushal + fork_trampoline），子进程 eax=0

· **exit(status) + wait(&status)**：标记 ZOMBIE → 唤醒父进程 → 僵尸收集 → 资源释放

修复的关键 Bug（6 个）：

1.栈地址共享（#UD）：pid 未初始化，所有线程用 stacks[-1]

2.proc_exit 上下文破坏：old 和 new 同指针，save 覆盖 next 上下文

3.kfree 静态栈：对 BSS 数组调用 kfree 导致堆损坏

4.退格无效：VGA 文本模式 \b 只显示字符图块

5.fork 子进程栈错误：memcpy 复制父进程内核栈 → switch_to ret 跳到随机地址

6.vmm_switch_pd 缺失：直接 mov cr3 不更新 current_pd_phys，导致后续映射写到错误页目录

实机验证通过： 完整链路 fork → child exit(42) → parent wait → 收集 status=42 → parent exit(0)，在 ASUS X542UF 实机上跑通。

3.6 M5: Syscall Linux ABI 兼容 + Shell 扩展

Commit: **5e7f319**

· 扩展系统调用接口，增强与 Linux i386 ABI 的兼容性

· Shell 功能扩展（命令解析增强、更多内建命令）

· 为 M6 文件系统阶段的 open/read/write/close 系统调用做准备

4 当前开发中：M5–M6 过渡阶段

当前分支 Wangmj 上的未提交更改显示正在进行 M5/M6 边界工作：
M5/M6 过渡期重点工作：

1.PCI 总线枚举：扫描 PCI 配置空间，查找 Class Code 0x010601（AHCI 控制器），获取 ABAR MMIO 基址

2.AHCI HBA 初始化：HBA Reset → Port 探测 → Command List / FIS 内存分配

3.ATA IDENTIFY：获取磁盘型号、序列号、LBA48 支持、容量

4.串口调试：添加 serial.cc 支持 COM1 输出，辅助磁盘驱动调试

文件/目录	说明
Makefile (修改)	添加 AHCI/PCI/Serial 编译目标
kernel/kernel.cc (修改)	集成 AHCI 初始化流程
include/kernel/drivers/ahci/ (新增)	AHCI 驱动头文件
include/kernel/drivers/pci/ (新增)	PCI 总线枚举头文件
include/kernel/lib/ (新增)	内核基础库头文件（serial 等）
kernel/drivers/ahci/ (新增)	AHCI HBA 初始化 + Port 管理 + ATA 命令
kernel/drivers/pci/ (新增)	PCI 配置空间扫描 + 设备枚举
kernel/lib/serial.cc/.h (新增)	串口调试输出支持

Table 3: 当前未提交更改

5 技术架构

5.1 系统架构总图



5.2 内存布局

5.3 中断向量分配

6 系统调用实现进度

目前已实现 7 个系统调用，占目标（约 50 个）的 14%。

7 关键 Bug 汇总

在 M1–M4 开发过程中，累计修复了 20 个实机 bug，以下为按阶段统计：
核心教训： x86 实机与 QEMU 模拟器在中断时序、I/O 延迟、BIOS 行为上存在显著差异。每次 QEMU 验证通过后必须进行实机验证，单靠模拟器不能保证正确性。

地址范围	大小	用途
0x00000000 – 0x000004FF	1.25 KB	BIOS 中断向量表 + BDA
0x00000500 – 0x000010FF	3 KB	Stage2 代码
0x00004000 – 0x00007FFF	16 KB	内核临时缓冲区
0x00007C00 – 0x00007DFF	512 B	MBR 启动扇区
0x00090000 – 0x0009FFFF	64 KB	内核栈
0x000A0000 – 0x000BFFFF	128 KB	VGA 显存
0x00100000+	—	内核代码/数据/堆
0x08000000 – 0x7FFFFFFF	~1.9 GB	可用物理内存
0xD0000000 – 0xE0000000	256 MB	内核堆虚拟地址空间

Table 4: 物理/虚拟内存布局

向量	中断源	说明
0x00 – 0x1F	CPU 异常	#DE, #UD, #DF, #GP, #PF 等 32 个
0x20	IRQ0 (PIT)	时钟中断 100Hz
0x21	IRQ1 (PS/2)	键盘中断
0x22 – 0x27	IRQ2–7	保留 / 级联
0x28 – 0x2F	IRQ8–15	从 PIC（RTC 等）
0x30 – 0x7F	保留	未来使用
0x80	int 0x80	系统调用 (DPL=3)

Table 5: 中断向量分配表

8 文件统计

9 后续规划

9.1 近期目标（M6: 磁盘驱动 + 文件系统）

1.PCI 总线枚举：扫描 Class Code 0x010601，获取 AHCI ABAR

2.AHCI HBA 初始化：HBA Reset → Port 探测 → Command List / FIS 内存分配

3.ATA IDENTIFY + READ/WRITE DMA EXT（LBA48 寻址）

4.块设备抽象接口：block_read / block_write

5.FAT32 解析：VBR BPB → FAT 表 → 根目录遍历

6.VFS 层：struct file + struct file_ops，每进程 fd table

7.设备文件：/dev/tty, /dev/sda, /dev/null

8.RAMFS：开发阶段不依赖磁盘的内存文件系统

编号	名称	功能	状态
1	exit	进程退出	已完成
3	read	读文件描述符	○ 待开始(M6)
4	write	写文件描述符 (调试)	已完成
5	open	打开文件	○ 待开始(M6)
6	close	关闭文件描述符	○ 待开始(M6)
11	execve	执行程序	○ 待开始(M7)
20	getpid	获取进程 PID	已完成
37	kill	发送信号	○ 待开始(M8)
41	dup	复制文件描述符	○ 待开始(M8)
42	pipe	创建管道	○ 待开始(M8)
45	brk	扩展进程堆	○ 待开始
54	ioctl	设备控制	○ 待开始(M6)
63	dup2	复制 fd 到指定编号	○ 待开始(M8)
90	mmap (old)	内存映射	○ 待开始
91	munmap	解除映射	○ 待开始
102	socketcall	socket 操作 (多路复用)	○ 待开始(M9)
120	clone	创建进程 (fork 底层)	已完成
252	exit_group	线程组退出	○ 待开始(M11)

Table 6: 系统调用实现进度

9.2 中期目标（M7–M9）

- **M7（用户态 + Shell）**：Ring 3 用户态切换、ELF 加载器、简化 libc、init 进程（PID 1）、交互式 Shell（REPL + 内建/外部命令）
- **M8（IPC + 管道）**：pipe + dup2 + 信号 + Shell 管道/重定向语法
- **M9（网络栈）**：RTL8111 驱动 + ARP + IPv4 + ICMP + TCP + Socket API

9.3 远期目标（M10–M11）

- **M10（ACPI 电源管理）**：RSDP → RSDT → FADT 解析 + S5 关机 + 重启
- **M11（SMP 多核）**：MADT 解析 + Local APIC 初始化 + AP 启动（INIT-SIPI-SIPI）+ 每核 runqueue + 大内核锁

9.4 终极验证目标

M11 完成后，以下操作应在 ASUS X542UF 实机上完整通过：

```
ohm$ ls /bin
ohm$ echo "hello" > /tmp/test.txt && cat /tmp/test.txt
ohm$ cat /etc/motd | wc -l
ohm$ ping 192.168.1.1
ohm$ /bin/vim /home/hello.txt
ohm$ /bin/g++ -o /tmp/a.out test.cc
ohm$ shutdown
```

阶段	Bug 数	典型问题
M1	4	GDT 基址错误、盘号硬编码、链接地址≠物理地址、清屏值错误
M2	4	ISR 栈帧偏移错误、PIT I/O 延迟、break code 未处理、清屏值错误
M3	6	io_wait 不兼容华硕 EC、E820 buffer 重叠、printk 格式不支持、内核扇区截断、Set 1/Set 2 扫描码、print_hex 低 4 位 bug
M4	6	pid 未初始化、proc_exit 上下文破坏、kfree 静态栈、退格无效、fork 栈错误、vmm_switch_pd 缺失
合计	**20**	

Table 7: 关键 Bug 统计

模块	文件数
boot/ (启动引导)	3 ASM + 2 LD
kernel/arch/x86/ (架构)	9 (CC + ASM + H)
kernel/mm/ (内存管理)	6 (CC + H)
kernel/proc/ (进程管理)	4 (CC + ASM + H)
kernel/drivers/ (驱动)	6+ (CC + H)
kernel/fs/ (文件系统)	5 (CC + H, 规划中)
kernel/lib/ (基础库)	4+ (CC + H)
user/ (用户态)	10+ (CC + ASM + H)
include/ (头文件)	20+
docs/ (文档)	6

Table 8: 源码文件统计（估算）

10 总结

Oh-my-os 项目已完成前 6 个里程碑（M0–M5），占总规划的 50%。内核已具备完整的启动链（BIOS → MBR → Stage2 → 保护模式 → 内核）、中断系统（PIC + IDT + PIT + PS/2 键盘）、内存管理（E820 + PMM + VMM + Heap + Demand Paging）、进程管理（PCB + RR 调度 + fork/exit/wait + Ring3 用户态）以及 Linux ABI 兼容的系统调用接口。所有阶段均在 ASUS X542UF 实机上验证通过，累计修复 20 个实机特有 bug。

当前正在进行 M5/M6 过渡阶段，开发重点为 AHCI 磁盘驱动、PCI 总线枚举和文件系统（FAT32 + VFS），这是项目中技术难度最高的模块。

关键数据：

· 已实现系统调用：7 个（目标 ~50 个）

· 实机验证：ASUS X542UF（i5-8250U, 8GB RAM, SATA SSD, Legacy BIOS）

· 平均 QEMU → 实机 bug 修复数：每阶段 4–6 个

· 代码规模：内核 ~3,000+ 行（C++/ASM），用户态 ~500+ 行